

Week 2 Report & Week 3 Plan

ML-Based Anomaly and Attack Detection for Smart-Meter Telemetry

Intern	Panayiotis Charalambous
Supervisor	Dr. Vasos Vassiliou
Date	25/06/2026

Part I — Week 2: Cleaning & Preprocessing

Week 2 turns the raw per-meter exports into analysis-ready hourly consumption series, and emits rule-detected data-integrity anomalies as standardized events. Work was developed and validated on the 5 available meters; the design scales to the full fleet unchanged.

Data-Handling Philosophy

- **Raw is immutable.** Files in `data/raw/` are never edited; every transformation reads from raw and writes elsewhere.
- **Processed is reproducible.** `data/processed/` holds cleaned outputs that can always be regenerated by re-running the pipeline.
- **Data is never committed.** Raw and processed data are git-ignored (sensitive utility data); the repository holds code and summary data reports only.
- **Design note.** Only the loading stage knows the data source (CSV today, database later); cleaning and all downstream stages are unaffected by that change.

Input Contract

`clean_meter()` consumes one loaded meter: a `DatetimeIndex` named `dateTime` (sorted ascending; the export is newest-first and sorted on load), with columns `deviceId`, `flow` (cumulative forward-flow counter, m^3), `signalCsq`, `isWaring`. As `flow` is cumulative, hourly consumption is obtained by differencing.

Cleaning Pipeline

The stages run in the following order; each lists what it does and why.

1. **Commissioning trim.** Drop the leading run before first real flow. *Why:* new meters report while powered-on but not yet plumbed in, producing leading zero-flow readings that are “not measuring,” not “zero usage.”
2. **Consumption differencing.** `consumption = flow.diff()`. *Why:* the counter is cumulative; the quantity of interest is per-hour usage.
3. **Counter-rollback handling (two-layer).** Where `consumption < 0` (counter decreased), set that hour to `NaN` and record a `counter_rollback` event. *Why:* a decrease is a reset/replace-ment/error, not negative usage (not logically possible); the corrupt value must not poison rolling statistics or feature scaling, but the rollback is a genuine anomaly so it is caught by a rule and emitted. We replace with `NaN` (not 0) because true usage during a reset is unknown.
4. **Hourly reindex.** Place the series on a continuous 1-hour grid; missing hours become `NaN`. *Why:* makes gaps explicit and locatable, and keeps the time axis regular so windowed features and rolling statistics behave correctly.
5. **Coverage-based service-start trim (scalable).** Compute reporting density (non-`NaN` fraction over a centred 7-day window); trim before the first point where density reaches a

threshold. *Why*: some meters report only sporadically for months before real service. A density rule (not a hardcoded date) is a *no-op on healthy meters* and scales identically to 2,031 meters.

6. **Usability classification.** From the post-trim reporting rate, label each meter **good** / **sparse** / **unusable**. *Why*: lets the pipeline auto-exclude hopeless meters at fleet scale rather than by hand.
7. **Gap-boundary diff guard.** A consumption value computed across a transmission gap is set to NaN. *Why*: on reconnection, the first reading reflects all water used during the silence; differencing would misattribute that whole chunk to one hour, producing a false spike. The hourly breakdown across a gap is unknowable, so it is nulled.

Event Output (rule-detected anomalies)

Rule-detected, deterministic anomalies are emitted now as OCSF-inspired JSONL records (one per line) matching the agreed schema, so the downstream Event Correlation Engine intern can validate ingestion early. Two event types are produced:

- **Counter rollback** — point anomaly; **observed_value** is the negative metering in cumulative flow.
- **Transmission gap** — collective anomaly; one event per meter per gap episode (a run of consecutive missing hours), carrying start, **end_time** and **duration_hours** in **context**. Sub-day gaps are ignored (meters batch-transmit $\sim$ twice daily, so short silences are routine).

Decision. Severity from manufacturer thresholds. Gap severity is anchored on the platform’s own alarm definitions: ≥ 72 h (3 days) $\rightarrow$ **medium** (“Sleeping Device”); ≥ 96 h (4 days) $\rightarrow$ **high** (“Communication Alarm”). This gives gaps a domain-validated severity scale rather than an arbitrary one.

Decision. Two event families (schema convention). *Value-based* anomalies (consumption spike, night burst) populate **observed_value** and **expected_value** as a comparable pair — the deviation between them is the anomaly. *Structural* anomalies (rollback, gap) have no comparable expected value, so both **observed_value** and **expected_value** are null for gaps; the gap’s defining quantity lives in **context.duration_hours**. This lets the engine interpret any event type consistently.

Decision. Per-meter event scope. The pipeline emits one event per meter; deciding that many meters gapping together is a single zone/network incident is the Event Correlation Engine’s job.

Data-Quality Findings

- **Coverage and behavior vary widely.** The 5 meters span ~ 11 – 14 months of hourly history; idle (zero-consumption) proportions range from $\sim 25\%$ to $\sim 96\%$. This wide variation is real behavioral signal and is the reason the Week 3 baseline must explicitly handle zero-inflation.
- **Counter rollbacks are rare** (0–5 per meter).
- **Gap-boundary artifact (found and fixed).** Meter ...0909 showed a lone $\sim 0.54 \text{ m}^3$ spike (April 2026) far above its normal ~ 0.01 – 0.1 m^3 : accumulated usage across a transmission gap collapsed into one hour by differencing. Fixed by the gap-boundary guard.
- **Meter ...1899 — sparse preamble, then low usage.** Reported almost nothing ($>99\%$ missing) for ~ 9 months before $\sim$ July 2025; the service-start trim removes this automatically. After trimming it is a clean ~ 11 -month series that is simply very low-usage ($\sim 94\%$ idle) — an extreme of the zero-inflation challenge and a likely “residential / low-use” clustering candidate, *not* a coverage problem (high *idle* rate $\neq$ high *missing* rate).

Week 2 Deliverables

- `src/cleaning.py` — reproducible cleaning with coverage-aware trimming and gap detection.
- `src/events.py` — rollback and gap event emission (OCSF-inspired JSONL) with validation.
- Cleaned series in `data/processed/`; per-meter data-quality report in `data_reports/`.
- `data/cleaning.md` — process, decisions, and findings (handover + report material).
- A real event sample shared with the Event Correlation Engine intern.

Part II — Week 3 Plan: Feature Engineering & Baseline Detector

Week 3 builds (a) a feature matrix per meter and (b) an interpretable baseline detector. The baseline’s purpose is to be the “compared-to-what?” reference that makes the Week 4–5 ML results (Isolation Forest, LOF) meaningful: if a simple rule matches the ML methods, the added complexity is not justified.

Feature Matrix

Plan. Build one feature matrix per meter (rows = hours, columns = features) from the cleaned consumption series. The four families below map onto the point / contextual / collective anomaly framework, which is why all four are needed.

- 1. Raw / instantaneous.** The value itself, `consumptiont`. Exposes gross point anomalies.
- 2. Statistical / windowed.** Rolling mean, std, min, max over windows (e.g. 24 h, 7 d), plus lag features (`consumptiont-1`, `t-24`, `t-168`). Provide the local context a value sits in; catch *collective* anomalies (a value fine alone but wrong for its neighbourhood).
- 3. Temporal / contextual.** `hour_of_day`, `day_of_week`, `is_weekend`, `is_holiday`. Encode *when* a reading occurred — the basis for *contextual* anomalies. Cyclical quantities are encoded so adjacent times are adjacent in feature space (hour 23 next to hour 0):

$$\text{hour_sin} = \sin\left(\frac{2\pi \text{hour}}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \text{hour}}{24}\right).$$

- 4. Profile-deviation.** The per-meter, per-(hour, day-of-week) expected value, giving the deviation of each reading from what is normal *for that meter at that time*. The most valuable contextual feature.

The Rolling z-Score and Why It Fails Here

The baseline’s core point-anomaly detector is the rolling z-score: how many standard deviations a value sits from its *local* mean,

$$z_t = \frac{x_t - \text{mean}(x_{t-w:t})}{\text{std}(x_{t-w:t})}, \quad \text{flag if } |z_t| > k \text{ (} k \approx 3 \text{)}.$$

Local (rolling) statistics are used because each meter’s “normal” drifts over time. On this data, the naive form breaks in three ways:

Failure 1 — zero-inflation (the dominant problem). With 25–96% zero hours, a window can be almost all zeros, giving mean ≈ 0 and std ≈ 0 . A perfectly normal small reading (e.g. 0.05 m^3) then yields $z = 0.05/0 \rightarrow \infty$, so the detector flags ordinary usage as a screaming anomaly.

Failure 2 — contamination. An anomaly *inside* the window inflates that window’s mean and std, partially masking itself and distorting neighbours.

Failure 3 — NaNs. Gaps and rollbacks are NaN; rolling operations must not silently produce garbage from half-missing windows.

Fixes We Will Use

Plan. Use *robust*, *lagging*, *floored* rolling statistics, NaN-aware throughout.

Robust statistics (median + MAD). Replace mean/std with median and the median absolute deviation, and use the modified z-score:

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|), \quad z_{\text{mod}} = \frac{x - \text{median}}{1.4826 \cdot \text{MAD}}$$

(the constant 1.4826 scales MAD to estimate the standard deviation for normal data; threshold ≈ 3.5). Median and MAD resist outliers, which fixes **Failure 2** as well.

Denominator floor. On windows that are more than half zeros, $\text{MAD} = 0$ and the modified z-score still diverges. We will impose a floor $\sigma_{\min}$:

$$z = \frac{x - \text{median}}{\max(1.4826 \cdot \text{MAD}, \sigma_{\min})},$$

so a near-zero window cannot manufacture an infinite score. Robust stats fix contamination; the floor fixes **Failure 1**; both are required.

Lagging window. We will compute statistics over the strict past, $x_{t-w:t-1}$ (excluding the current point), so a point cannot contaminate its own baseline.

NaN-aware operations. We will use `min_periods` and NaN-aware reductions so missing hours (**Failure 3**) do not corrupt a window.

The Per-Meter Profile (for the deviation check)

Plan. Build, per meter, a $24 \times 7 = 168$ -cell lookup table: for each (hour-of-day, day-of-week) bucket, the *median* consumption over the meter’s own history.

The profile is the meter’s weekly fingerprint — its typical usage for, say, “Tuesday 3 am.” The deviation feature is then

$$\text{deviation}_t = \text{consumption}_t - \text{profile}[\text{hour}(t), \text{dow}(t)].$$

dow = day of the week

Median (not mean) per bucket keeps the expected value robust to occasional bursts, so real anomalies stand out instead of inflating the baseline. This is what turns context-blind detection into contextual detection — a 0.4 m^3 reading is normal at 8 am but a night-time-burst anomaly at 3 am. The same 168-vector later seeds the behavioral clustering (validated against true customer category - if we get access to the EOA MySQL DB as described in last week’s report).

Full Baseline Bundle

Plan. The baseline is a small bundle of interpretable checks, each targeting an anomaly category the others miss. Each produces a continuous *score* (kept un-thresholded for fair AUC comparison against the ML methods in Week 8) plus a flag.

Check	Category	What it catches
Robust rolling z-score (median/MAD, floored, lagging)	Point	Spikes and drops vs the meter's recent local normal
Profile-deviation check	Contextual	Normal-magnitude usage at an abnormal time (e.g. night-time burst)
Profile-aware stuck check	Collective	Flat when the profile says this bucket should be active (not merely flat — meters are legitimately idle)
Gap / missing check	Collective	Runs of missing hours (largely from cleaning; emitted as gap events)
Manufacturer rules	Contextual / point	Manufacturer-validated thresholds, e.g. Logic Leakage: consumption >10 L in both 02:00–03:00 and 03:00–04:00 (Manufacturer rules are further described in last week's report)

Plan. Order of build: (1) feature matrix; (2) per-meter profile; (3) robust rolling z-score; (4) profile-deviation and profile-aware stuck checks; (5) wire in the gap check and manufacturer rules; (6) keep all scores continuous for Week 8 evaluation. The profile is built first among the detectors, since the deviation and stuck checks both depend on it.